第三章：栈和队列

作业题目重建与超详细解析

题目 3.1

回文判定：利用栈比较字符串的镜像位置

题目 3.2

双端循环队列：队尾删除与队头插入

结构：每道题均按“原题 → 思路 → 算法/程序 → 正确性 → 示例 → 复杂度 → 易错点”连续排版。

目录

1 题目 3.1：回文判定 2

2 题目 3.2：双端循环队列 6

1 题目 3.1：回文判定

原题重建

回文是指正读反读均相同的字符序列，如“abba”和“abdba”均是回文，但“good”不是回文。试写一个算法程序判定给定的字符串是否为回文。（提示：将一半字符入栈）

答案与详细解析

1. 为什么用栈可以判断回文

栈（stack）的核心性质是**后进先出（LIFO）**：最后压入的字符最先弹出。若把字符串的前一半从左到右依次压入栈，那么弹栈时得到的顺序恰好是前一半的逆序。随后只要把它与字符串后一半从左到右逐字符比较，就相当于检查左右两侧的镜像字符是否完全相等。

对长度为 n 的字符串 s：

- 前半部分下标为 $0, 1, \ldots, \lfloor n/2 \rfloor - 1$，全部入栈；
- 若 n 为奇数，中间字符不影响回文性，因此直接跳过；
- 后半部分从下标 $\lceil n/2 \rceil$ 开始，与栈顶逐个比较；
- 任意一次不相等，都可以立即判定“不是回文”；若全部相等，则为回文。

下标起点统一公式

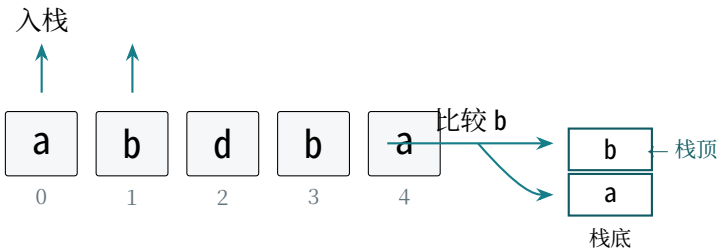
后一半的起始下标可以直接写成

$$\boxed{j = \frac{n+1}{2}}$$

这里使用整数除法。若 n 为偶数，例如 $n = 4$，得到 $j = 2$；若 n 为奇数，例如 $n = 5$，得到 $j = 3$，自动跳过中间位置 2。

2. 过程示意：以 abdba 为例

字符串长度 $n = 5$，前一半有 $\lfloor 5/2 \rfloor = 2$ 个字符：a, b。



先比较后半部分第一个字符 b 与栈顶 b，相等后弹栈；再比较最后一个字符 a 与新的栈顶 a，仍相等。因此 abdba 是回文。

3. 教材风格算法（按题目提示使用栈）

用伪代码表示如下。为了突出算法本身，假设 Push、Pop、InitStack 已经按栈的基本操作实现。

Listing 1: 回文判定：栈算法伪代码

```
bool Palindrome(const char s[], int n) {
    Stack S;
    InitStack(S);

    for (int i = 0; i < n / 2; ++i)
        Push(S, s[i]);

    for (int j = (n + 1) / 2; j < n; ++j) {
        char x;
        Pop(S, x);
        if (x != s[j])
            return false;
    }

    return true;
}
```

4. 可直接编译运行的 C++ 程序

下面使用标准库 `std::stack<char>` 实现，逻辑与上面的教材算法完全一致。

Listing 2: C++ 完整程序：使用一半字符入栈

```cpp
#include <iostream>
#include <stack>
#include <string>
using namespace std;

bool isPalindrome(const string& s) {
    stack<char> st;
    int n = static_cast<int>(s.size());

    for (int i = 0; i < n / 2; ++i)
        st.push(s[i]);

    for (int i = (n + 1) / 2; i < n; ++i) {
        if (st.empty() || st.top() != s[i])
            return false;
        st.pop();
    }

    return st.empty();
}

int main() {
    string s;
    cin >> s;

    if (isPalindrome(s))
        cout << "YES\n";
    else
        cout << "NO\n";

    return 0;
}
```

5. 三个样例逐步验证

字符串	入栈字符	后半部分比较过程	结果
abba	a,b	b 与栈顶 b 相等；a 与栈顶 a 相等	回文
abdba	a,b	跳过中间 d；b 与 b 相等；a 与 a 相等	回文
good	g,o	o 与栈顶 o 相等；d 与栈顶 g 不等	非回文

6. 正确性说明

设前半部分依次为 $s_0, s_1, \ldots, s_{k-1}$，其中 $k = \lfloor n/2 \rfloor$。它们依次入栈后，第一次弹出的字符一定是 s_{k-1}，第二次是 s_{k-2}，依此类推。因此，与后一半从左到右比较时，比较的恰好是一对对镜像位置：

$$(s_{k-1},\ s_{\lceil n/2 \rceil}),\quad (s_{k-2},\ s_{\lceil n/2 \rceil+1}),\quad \ldots$$

如果字符串是回文，每一对镜像字符必然相等，算法不会提前返回 `false`；反过来，只要存在一对镜像字符不相等，算法就会在对应比较处发现不等并返回 `false`。因此，该算法与回文定义等价。

7. 复杂度分析

对大约一半字符执行入栈，再对另一半字符执行比较与出栈，所以时间复杂度为

$$\boxed{O(n)}.$$

栈中最多保存 $\lfloor n/2 \rfloor$ 个字符，因此辅助空间复杂度为

$$\boxed{O(n)}.$$

若严格写出常数，则最多使用约 $n/2$ 个字符的额外存储，但渐进复杂度仍记为 $O(n)$。

8. 一题多解：双指针法（用于理解与对照）

若题目**没有**要求使用栈，最直接的方法是设置左右两个下标 $i = 0$、$j = n - 1$，每次比较 $s[i]$ 与 $s[j]$，相等则同时向中间移动，不等立即结束。这样仍为 $O(n)$ 时间，但只需 $O(1)$ 额外空间。

Listing 3: 对照解法：双指针

```cpp
bool isPalindrome2(const string& s) {
    int i = 0;
    int j = static_cast<int>(s.size()) - 1;
    while (i < j) {
        if (s[i] != s[j])
            return false;
        ++i;
        --j;
    }
    return true;
}
```

易错点

1. **奇数长度不要比较中间字符**。中间字符没有镜像搭档，应从 $(n+1)/2$ 开始比较后一半。
2. **不要把整个字符串都入栈再与原串比较**。虽然也能做，但题目明确提示“将一半字符入栈”，半栈方案更直接、空间也少一半。
3. **字符编码问题**。本题示例均为英文字母，`char` 足够。若处理 UTF-8 中文文本，一个汉字通常占多个字节，不能直接把单个字节当作一个“字符”比较。
4. **空串的约定**。按数学与大多数算法约定，空串可视为回文；若课程另有规定，应按教师给定的输入范围处理。

9. 权威核验

OpenDSA 的栈章节将栈定义为 LIFO（后进先出），并给出 `push`/`pop`/`top` 等基本操作，这正是本题“前半段入栈、后半段逆序比较”的依据。参见：OpenDSA: Stacks and Queues。

2 题目 3.2：双端循环队列

原题重建

如果允许在循环队列的两端都可以进行插入和删除操作。要求：

① 写出循环队列的类型定义；

② 写出“从队尾删除”和“从队头插入”的算法程序。

答案与详细解析

1. 先统一 front / rear 的含义

循环队列有多种合法的指针约定。为了让公式最清楚，本解答采用数据结构课程中非常常见的一种：

- `front`：指向**队头元素**；
- `rear`：指向**队尾元素的下一个空位置**；
- 队空：`front == rear`；
- 为区分“队空”和“队满”，数组始终保留一个空单元；
- 队满：`(rear + 1) % MAXQSIZE == front`。

因此，若数组长度为 `MAXQSIZE`，实际最多存放 `MAXQSIZE - 1` 个元素。

循环下标的本质

普通数组下标“走一步”是 $i+1$；循环数组中“走一步”必须取模：

$$\mathrm{next}(i) = (i+1) \bmod M.$$

反向“退一步”应写成

$$\boxed{\mathrm{prev}(i) = (i-1+M) \bmod M},$$

其中 $M =$ `MAXQSIZE`。加上 M 是为了避免 C/C++ 中出现负下标。

2. 循环队列类型定义

Listing 4: 双端循环队列的顺序存储类型定义

```
const int MAXQSIZE = 100;
typedef int QElemType;

typedef struct {
    QElemType data[MAXQSIZE];
    int front;
    int rear;
} CirDeque;
```

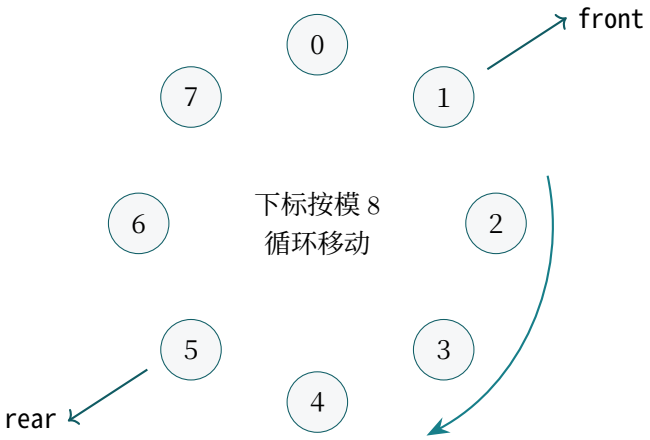
初始化只需令两者相等:

Listing 5: 初始化

```
void InitDeque(CirDeque& Q) {
    Q.front = 0;
    Q.rear = 0;
}
```

若教材把数组写成动态空间 `QElemType *base`，其 `front`/`rear` 的移动公式完全相同；这里只是用定长数组让程序更完整、便于直接阅读。

3. 环形结构示意



图中编号只是为了说明“到达末端后回到 0”。实际操作不需要移动数组元素，只移动 `front`/`rear` 下标，因此队列两端操作都可以做到常数时间。

4. 从队尾删除：DeleteRear

因为 `rear` 指向“队尾元素的下一个空位置”，所以不能直接读取 `data[rear]`。正确顺序是:

1. 先判断队空;
2. 将 `rear` 反向移动一格，使它落到原队尾元素上;
3. 读出该元素。

Listing 6: 从队尾删除

```
bool DeleteRear(CirDeque& Q, QElemType& e) {
    if (Q.front == Q.rear)
        return false;

    Q.rear = (Q.rear - 1 + MAXQSIZE) % MAXQSIZE;
    e = Q.data[Q.rear];
    return true;
}
```

为什么必须“先 rear−，再取元素”

假设当前队列中的有效元素占用下标 $2,3,4$，则 front=2，rear=5。真正的队尾元素在下标 4，而下标 5 是下一个可插入位置。因此删除队尾时必须先把 rear 从 5 退到 4，再读取 data[4]。

5. 从队头插入：InsertFront

在队头前方插入新元素时，新的队头应位于“原 front 的前一个循环位置”。因此正确顺序是：

1. 先判断队满；
2. 将 front 反向移动一格；
3. 把新元素写到新的 front 位置。

Listing 7: 从队头插入

```
bool InsertFront(CirDeque& Q, QElemType e) {
    if ((Q.rear + 1) % MAXQSIZE == Q.front)
        return false;

    Q.front = (Q.front - 1 + MAXQSIZE) % MAXQSIZE;
    Q.data[Q.front] = e;
    return true;
}
```

最容易写错的一行

不能写成

$$\texttt{Q.front = (Q.front - 1) \% MAXQSIZE;}$$

因为当 front=0 时，C/C++ 中 -1 % MAXQSIZE 仍可能得到负值，随后访问 data[-1] 就越界了。正确写法是

$$\boxed{\texttt{(Q.front - 1 + MAXQSIZE) \% MAXQSIZE}}.$$

6. 用一个具体队列走一遍

设 MAXQSIZE=8，当前有效元素依次为 $20, 30, 40$，并且

$$\texttt{front} = 2, \qquad \texttt{rear} = 5,$$

即数组下标 $2, 3, 4$ 分别存放 $20, 30, 40$。

（1）执行 DeleteRear：

$$\texttt{rear} : 5 \to (5 - 1 + 8) \bmod 8 = 4,$$

于是返回 data[4]=40。删除后剩余队列为 $20, 30$，此时 front=2、rear=4。

（2）随后执行 InsertFront(Q, 10)：

$$\texttt{front} : 2 \to (2 - 1 + 8) \bmod 8 = 1,$$

再令 data[1]=10。于是队列从队头到队尾变为

$$\boxed{10, 20, 30}.$$

若原来 front=0，插入队头时则有

$$(0 - 1 + 8) \bmod 8 = 7,$$

这正体现了“循环”——下标从 0 的前面绕回数组末端 7。

7. 与普通队列四个端点操作的关系

操作	指针变化	数据读写位置
队尾插入	先写后 rear=(rear+1)%M	写 data[rear]
队头删除	先读后 front=(front+1)%M	读 data[front]
队尾删除	**先 rear=(rear-1+M)%M**	**再读 data[rear]**
队头插入	**先 front=(front-1+M)%M**	**再写 data[front]**

从这个表可以看出, 题目新增的两个操作本质上就是把普通循环队列的指针移动方向反过来。允许四种操作后，该结构就是**双端队列（deque, double-ended queue）**。

8. 正确性与复杂度

DeleteRear 的正确性： rear 始终指向队尾后一格，所以它反向移动一次后必然落在当前队尾元素上；读取后，新 rear 又恰好成为删除后队尾的下一空位，因此不变量保持不变。

InsertFront 的正确性： front 始终指向队头元素。将它反向移动一次得到队头之前的一个空位，并把新元素写入后，新 front 正好指向新队头，因此不变量同样保持不变。

两种操作都只做有限次下标运算、判断和一次读/写，不移动其他元素，因此时间复杂度均为

$$\boxed{O(1)}.$$

顺序存储数组需要 $O(M)$ 空间，其中 $M = \texttt{MAXQSIZE}$。

9. 可直接整理到作业中的核心代码

如果只保留题目真正要求的“类型定义 + 队尾删除 + 队头插入”，可以整理成下面这一段。它与前面的详细推导使用完全相同的 front/rear 约定。

Listing 8: 题目 3.2 核心答案汇总

```
const int MAXQSIZE = 100;
typedef int QElemType;

typedef struct {
    QElemType data[MAXQSIZE];
    int front;
    int rear;
} CirDeque;

bool DeleteRear(CirDeque& Q, QElemType& e) {
    if (Q.front == Q.rear)
        return false;
    Q.rear = (Q.rear - 1 + MAXQSIZE) % MAXQSIZE;
    e = Q.data[Q.rear];
    return true;
}

bool InsertFront(CirDeque& Q, QElemType e) {
    if ((Q.rear + 1) % MAXQSIZE == Q.front)
        return false;
    Q.front = (Q.front - 1 + MAXQSIZE) % MAXQSIZE;
    Q.data[Q.front] = e;
    return true;
}
```

易错点汇总

1. **front/rear 定义必须先统一**。有些教材令 rear 指向“最后一个元素”而不是“最后元素的下一空位”，此时公式会不同；不要混用两套约定。
2. **本解答采用“牺牲一个存储单元”区分队满与队空**。因而容量为 MAXQSIZE-1。也可以额外设置 size 或标志位来使用全部数组空间，但判满公式要相应改变。
3. **删除队尾要先移动 rear，再取值；队头插入要先移动 front，再赋值**。顺序颠倒会访问错误的位置。
4. **反向取模要加 MAXQSIZE**。这是 C/C++ 环形数组最常见的边界错误之一。

10. 权威核验

OpenDSA 的循环队列实现说明了通过取模运算让 `front`/`rear` 在数组中循环移动，并采用额外空位区分队满与队空；这与本文采用的顺序循环实现原则一致。参见：OpenDSA: Queues - The Circular Queue。

C++ 标准库文档将 `std::deque` 定义为 double-ended queue，并提供 `push_front` 与 `pop_back` 等两端常数时间操作。该文档可用于核验“允许两端插入、删除即双端队列”这一抽象定义；但标准库 `std::deque` 的内部实现并不要求采用单个循环数组。参见：cppreference: std::deque。

核验结论：两题算法与公开数据结构教材中栈的 LIFO 性质、循环队列的模运算规则，以及双端队列的两端操作定义一致。本文中的 `front`/`rear` 具体语义已在题 3.2 开头明确，因此所有公式在同一约定下自洽。